

Cálculo Numérico Aplicado

Atividade e Programa para casa #3

Prof. Doutor Rafael Gabler Gontijo

7 de maio de 2026

O que é um sistema linear?

Um sistema de equações algébricas lineares de n variáveis pode ser organizado em uma estrutura matricial do tipo $[A]_{n,n}\{x\}_n = \{b\}_n$ em que os subíndices indicam a ordem de cada matriz ou vetor. Uma forma geral de representarmos um sistema linear pode ser dada por

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}. \quad (1)$$

Nessa estrutura da equação (1) dizemos que $\mathbf{A}$ é a matriz dos coeficientes, $\mathbf{x}$ é o vetor solução e $\mathbf{b}$ é um vetor associado a termos de fonte. Estruturas matemáticas com essa característica são chamadas de sistemas lineares e aparecem em diversos problemas do mundo real. Toda simulação computacional desenvolvida para modelagem de problemas de engenharia tem dentro de seu código fonte alguma chamada para uma função, subrotina ou biblioteca cuja única função dentro desse código consiste em resolver um sistema linear. Podemos dizer que sistemas lineares são uma espécie de *blocos construtores* no campo dos métodos numéricos, uma vez que costumam aparecer em tópicos como otimização, regressão e soluções de equações diferenciais ordinárias e parciais.

Como estratégia de solução de sistemas do tipo, temos tanto métodos diretos quanto iterativos. A mãe de todos os métodos diretos é a eliminação Gaussiana. É desta técnica que derivam-se esquemas como Decomposição LU, Cholesky e o algoritmo de Thomas. Veremos a seguir uma breve apresentação da técnica de eliminação Gaussiana.

A eliminação Gaussiana

A ideia do algoritmo de eliminação Gaussiana consiste em dividir a estratégia de solução em duas etapas. Na etapa 1, chamada de *eliminação progressiva*, transformamos o sistema linear num sistema triangular (superior ou inferior). Na etapa 2, que chamamos de *substituição regressiva*, partimos de baixo (ou de cima, dependendo de como optamos por fazer a transformação na etapa 1), e por meio de uma série de substituições regressivas vamos estimando cada valor de $\{x\}$. A ideia fica mais clara com imagens. A transformação de um sistema linear do tipo $[A]\{x\} = \{b\}$ em um sistema triangular superior na etapa de eliminação leva a um sistema do tipo:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & a'_{22} & a'_{23} & a'_{24} \\ 0 & 0 & a''_{33} & a''_{34} \\ 0 & 0 & 0 & a'''_{44} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} b_1 \\ b'_2 \\ b''_3 \\ b'''_4 \end{pmatrix}. \quad (2)$$

Note que aqui tanto os termos da matriz $[A]$ quanto do vetor $\{b\}$ sofrem alterações ao longo do processo de transformação, porém a solução $\{x\}$ do sistema modificado é a mesma do sistema original. Essa transformação do sistema original em um sistema triangular sem alteração da solução é a grande ideia por trás do método da eliminação Gaussiana. Uma vez que tenhamos triangularizado o sistema, podemos começar resolvendo ele de baixo

para cima para obter:

$$x_4 = \frac{b_4'''}{a_{44}'''}, \quad (3)$$

$$x_3 = (b_3'' - a_{34}''x_4)/a_{33}'', \quad (4)$$

$$x_2 = (b_2' - a_{23}'x_3 - a_{24}'x_4)/a_{22}', \quad (5)$$

$$x_1 = (b_1 - a_{12}x_2 - a_{13}x_3 - a_{14}x_4)/a_{11}. \quad (6)$$

Essa última etapa é o que chamamos de *substituição regressiva*. Mais para frente voltaremos à questão da palavra *ingênua* no título dessa seção. Por enquanto, o que estamos buscando para entender melhor como realizar essas duas etapas na prática, precisamos construir o algoritmo por meio de um exemplo. Considere então um sistema linear geral de ordem n , expresso por

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \quad (7)$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2 \quad (8)$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n \quad (9)$$

Para esse sistema geral, apresentaremos a seguir uma espécie de receita de bolo para a construção do algoritmo de eliminação Gaussiana.

Passo a passo do algoritmo de Eliminação Gaussiana

1. Multiplique (7) por a_{21}/a_{11} :

$$a_{21}x_1 + \left(\frac{a_{21}}{a_{11}}\right)a_{12}x_2 + \left(\frac{a_{21}}{a_{11}}\right)a_{13}x_3 + \cdots + \left(\frac{a_{21}}{a_{11}}\right)a_{1n}x_n = \left(\frac{a_{21}}{a_{11}}\right)b_1; \quad (10)$$

2. Subtraia (11) de (8):

$$\left(a_{22} - \frac{a_{21}a_{12}}{a_{11}}\right)x_2 + \cdots + \left(a_{2n} - \frac{a_{21}a_{1n}}{a_{11}}\right)x_n = \left(b_2 - \frac{a_{21}b_1}{a_{11}}\right), \quad (11)$$

que pode ser reescrita como

$$a'_{22}x_2 + \cdots + a'_{2n}x_n = b'_2; \quad (12)$$

3. Aplica-se o mesmo procedimento para as linhas subsequentes. Por exemplo, multiplicando (7) por a_{31}/a_{11} e subtraindo o resultado da terceira linha do sistema, eliminamos o coeficiente vinculado à x_1 dessa linha também. Nesse procedimento, a equação (7) é chamada de equação pivô e o termo a_{11} é denominado *coeficiente ou elemento pivô*. Aplicando esse procedimento para as demais linhas da matriz nos leva a uma estrutura do tipo:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \quad (13)$$

$$a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n = b'_2 \quad (14)$$

$$a'_{32}x_2 + a'_{33}x_3 + \cdots + a'_{3n}x_n = b'_3 \quad (15)$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots$$

$$a'_{n2}x_2 + a'_{n3}x_3 + \cdots + a'_{nn}x_n = b'_n \quad (16)$$

4. Precisamos agora eliminar x_2 da equação (15) em diante. Para isso, multiplicamos a equação (14) pelo fator a'_{32}/a'_{22} e subtraímos o resultado de (15), transformando então a equação (15) em:

$$a''_{33}x_3 + \cdots + a''_{3n}x_n = b''_3, \quad (17)$$

aplicando esse passo para todas as linhas abaixo da equação (15), transformamos nosso sistema em

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1$$

$$a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n = b'_2$$

$$a''_{33}x_3 + \cdots + a''_{3n}x_n = b''_3$$

$$\vdots \quad \quad \quad \vdots \quad \quad \quad \vdots$$

$$a'_{n3}x_3 + \cdots + a'_{nn}x_n = b'_n$$

5. Continuamos o processo para irmos eliminando x_3 a partir da linha 4 em diante e assim sucessivamente, até que nosso sistema tenha se tornado um sistema triangular superior do tipo

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1$$

$$a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n = b'_2$$

$$a''_{33}x_3 + \cdots + a''_{3n}x_n = b''_3$$

$$\vdots \quad \quad \quad \vdots \quad \quad \quad \vdots$$

$$a^{(n-1)}_{nn}x_n = b^{(n-1)}_n.$$

Nessa simbologia o número de aspas representa o número de modificações pelas quais aquela linha passou durante o processo de eliminação. O expoente $(n-1)$ nos coeficientes da última linha do sistema indica que essa linha passou por $n-1$ modificações ao longo do caminho de triangularização do sistema.

6. O último passo para resolvermos nosso sistema, consiste na aplicação da etapa de substituição regressiva. Para isso, começamos calculando x_n com

$$x_n = \frac{b^{(n-1)}_n}{a^{(n-1)}_{nn}}; \quad (18)$$

7. Em seguida, iteramos dentro do seguinte laço:

$$x_i = \frac{b^{(i-1)}_i - \sum_{j=i+1}^n a^{(i-1)}_{ij}x_j}{a^{(i-1)}_{ii}} \rightarrow i = n-1, n-2, \dots, 1. \quad (19)$$

Decomposição L.U

Uma outra estratégia possível de solução de um sistema linear, baseada no método da eliminação Gaussiana consiste em triangularizar o sistema linear tanto por cima quanto por baixo. Em outras palavras, decompor a matriz dos coeficientes em duas matrizes triangulares, uma superior $[U]$ e outra inferior $[L]$, sem precisar manipular o vetor $\{b\}$ durante o processo de eliminação e solução. Essa técnica é conhecida como decomposição L.U e será apresentada a seguir.

A ideia geral da decomposição L.U se baseia no seguinte racional:

- Nosso sistema linear original é dado por

$$[A]\{x\} = \{b\} \rightarrow [A]\{x\} - \{b\} = \{0\} \quad (20)$$

- Suponha agora que a equação (20) possa ser escrita em termos de um sistema triangular superior, como se estivéssemos implementado a etapa de eliminação do algoritmo de eliminação Gaussiana descrito aqui, de tal sorte que

$$[U]\{x\} = \{d\} \rightarrow [U]\{x\} - \{d\} = \{0\} \quad (21)$$

- Agora, suponha a existência de uma matriz triangular inferior com 1's na diagonal, que chamaremos aqui de $[L]$. Se multiplicarmos $[L]$ pela equação (21) teremos

$$[L]([U]\{x\}) = [L]\{d\} \rightarrow [L]([U]\{x\} - \{d\}) = \{0\} \quad (22)$$

- Como o lado direito da equação (22) é igual o lado direito da equação (20) podemos fazer

$$[L]([U]\{x\} - \{d\}) = [A]\{x\} - \{b\} \quad (23)$$

- Comparando os termos da equação (23) concluímos que:

$$[A] = [L][U] \quad (24)$$

e

$$[L]\{d\} = \{b\} \quad (25)$$

Baseado nesses procedimentos, a ideia do algoritmo de decomposição L.U consistem em

1. Decompor $[A]$ em $[L]$ e $[U]$;
2. Calcular $\{d\}$ por substituição progressiva, usando $[L]\{d\} = \{b\}$. Note que isso só é possível pelo fato de $[L]$ ser uma matriz triangular inferior;
3. Com o valor de $\{d\}$ calculamos $\{x\}$ por substituição regressiva utilizando $[U]\{x\} = \{d\}$, o que só é possível pelo fato de $[U]$ ser uma matriz triangular superior.

A grande pergunta aqui é: como decompor $[A]$ em $[L]$ e $[U]$? E a resposta para essa pergunta está no próprio algoritmo de eliminação Gaussiana. Quando aplicamos o algoritmo de eliminação Gaussiana para triangularizar o sistema, acabamos jogando fora os fatores utilizados na etapa de eliminação. Esses fatores, se armazenados nas posições adequadas em uma outra matriz acabam dando origem à matriz $[L]$ quando usados para triangularizar a matriz $[A]$ na matriz $[U]$.

Para exemplificar o procedimento vamos aplicar a estratégia de decomposição L.U para um sistema 3×3 genérico, dado por:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}. \quad (26)$$

A ideia aqui é aplicar a eliminação Gaussiana apenas sobre a matriz $[A]$ sem manipular $\{b\}$ para obter $[U]$ e armazenar os fatores utilizados no processo sobre uma matriz identidade $[I]$ para formar a matriz $[L]$. Do ponto de vista de sequência de procedimentos fazemos:

1. Calcula-se o fator $f_{21} = a_{21}/a_{11}$;
2. Multiplica-se a linha 1 por f_{21} e subtrai-se o resultado da linha 2:

$$[U] = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \quad (27)$$

3. Armazenamos o fator f_{21} na posição ℓ_{21} de uma matriz $[I]$ para começarmos a construir $[L]$:

$$[L] = \begin{pmatrix} 1 & 0 & 0 \\ f_{21} & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (28)$$

4. Calcula-se agora o fator $f_{31} = a_{31}/a_{11}$;
5. Multiplica-se a linha 1 por f_{31} e subtrai-se o resultado da linha 3:

$$[U] = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & a'_{32} & a'_{33} \end{pmatrix} \quad (29)$$

6. Armazenamos o fator f_{31} na posição ℓ_{31} de $[L]$:

$$[L] = \begin{pmatrix} 1 & 0 & 0 \\ f_{21} & 1 & 0 \\ f_{31} & 0 & 1 \end{pmatrix} \quad (30)$$

7. Calcula-se agora o fator $f_{32} = a'_{32}/a'_{22}$;
8. Multiplica-se a linha 2 por f_{32} e subtrai-se o resultado da linha 3:

$$[U] = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & 0 & a''_{33} \end{pmatrix} \quad (31)$$

9. Armazenamos o fator f_{32} na posição ℓ_{32} de $[L]$:

$$[L] = \begin{pmatrix} 1 & 0 & 0 \\ f_{21} & 1 & 0 \\ f_{31} & f_{32} & 1 \end{pmatrix} \quad (32)$$

Esse procedimento decompõe $[A]$ em $[A] = [L][U]$ e não envolve operações em $\{b\}$, nesse sentido ele tem um custo computacional ligeiramente menor do que o custo da mesma etapa no processo original de eliminação Gaussiana, gastando apenas um pouco mais de memória para armazenar os fatores na matriz $[L]$.

Em termos de terminologia, existem dois tipos de métodos de decomposição L.U. Quando as matrizes $[L]$ e $[U]$ possuem as características descritas na equação (33) dizemos que o procedimento é denominado *decomposição de Doolittle ou fatoração L.U.*

$$[L] = \begin{pmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{pmatrix} \quad \text{e} \quad [U] = \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix} \quad (33)$$

Quando as matrizes $[L]$ e $[U]$ possuem as características descritas na equação (34) dizemos que o procedimento é denominado *decomposição de Crout*.

$$[L] = \begin{pmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix} \quad \text{e} \quad [U] = \begin{pmatrix} 1 & u_{12} & u_{13} \\ 0 & 1 & u_{23} \\ 0 & 0 & 1 \end{pmatrix} \quad (34)$$

Algoritmo de Thomas para matrizes tridiagonais (TDMA)

Uma matriz de banda é aquela para a qual todos os termos são nulos com exceção da diagonal e de uma faixa centrada em torno da diagonal, como a que vemos na equação (35)

$$[A] = \begin{pmatrix} a_{11} & a_{12} & 0 & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ 0 & a_{32} & a_{33} & a_{44} \\ 0 & 0 & a_{43} & a_{44} \end{pmatrix}, \quad (35)$$

para a matriz apresentada em (35) temos uma largura de banda igual a 3, o que significa que a maior quantidade de termos não nulos por linha é 3. Na verdade, essa é a maior largura de banda possível para uma matriz de banda 4×4 . Entretanto, podemos ter matrizes de ordem bem mais alta com essa mesma largura de banda, como a matriz apresentada em (36)

$$[A] = \begin{pmatrix} f_1 & g_1 & 0 & 0 & \cdots & 0 \\ e_2 & f_2 & g_2 & 0 & \cdots & 0 \\ 0 & e_3 & f_3 & g_3 & \cdots & 0 \\ 0 & 0 & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & e_{n-1} & f_{n-1} & g_{n-1} & 0 \\ \cdots & \cdots & \cdots & e_n & f_n & g_n \end{pmatrix}, \quad (36)$$

note que para a matriz tridiagonal $n \times n$ representada em (35) podemos armazenar todos os termos não nulos em termos dos vetores e, f, g , cada qual contendo a seguinte quantidade de termos:

- $e = [2, \cdots, n] \rightarrow n - 1$ termos
- $f = [1, \cdots, n] \rightarrow n$ termos

- $g = [1, \dots, n] \rightarrow n$ termos

de tal sorte que ao invés de armazenarmos n^2 termos, podemos armazenar apenas $3n - 1$ termos. Isso gera uma grande economia de espaço de armazenamento. A aplicação da técnica de decomposição L.U para resolver esse tipo de sistema pode ser implementada em termos de um algoritmo especial, denominado *algoritmo de Thomas* que resolve o sistema por meio do pseudocódigo em FORTRAN apresentado em (1). Note que ao invés de vários laços entrelaçados temos apenas 3 laços simples em sequência, o que gera uma enorme economia de recursos computacionais para a solução do sistema. Na verdade, o que o algoritmo de Thomas faz é basicamente a aplicação da decomposição L.U, porém já direcionada a matrizes tridiagonais, de tal sorte que ao invés de trabalhar com as n^2 componentes originais da matriz $[A]$, o mesmo trabalha apenas com os subvetores e, f, g necessários para a escrita da matriz $[A]$ no limite em que a mesma é uma matriz de largura de banda igual a 3. Além disso, no pseudocódigo apresentado, o vetor $\{b\}$, associado aos termos de fonte, está representado em termos de um vetor $\{r\}$. A pergunta central aqui é: *qual a diferença entre a decomposição L.U e o método de Thomas?* A resposta é: conceitualmente nenhuma, o que muda é apenas o algoritmo.

```

! Decomposição L.U
do k=2,n
e(k)= e(k)/f(k-1)
f(k) = f(k) - e(k)*g(k-1)
end do

!Substituição progressiva = cálculo de {d}
do k=2,n
r(k)=r(k) - e(k)*r(k-1)
end do

!Substituição regressiva = cálculo de {x}
x(n) = r(n)/f(n)
do k= n-1,1
x(k)=(r(k) - g(k)*x(k+1))/f(k)
end do

```

Figura 1: Pseudocódigo em FORTRAN para implementação do algoritmo de Thomas

Sistemas envolvendo matrizes simétricas

No contexto de sistemas lineares envolvendo matrizes simétricas, uma das técnicas de solução consiste na decomposição ou fatoração de Cholesky. Essa técnica se deve ao cartógrafo francês André-Louis Cholesky e consiste na proposição de relações simples de recorrência para decompor uma matriz $[A]$ simétrica no produto entre duas matrizes triangulares na forma $[A] = [L][L]^T$. Para apresentar a ideia da decomposição de Cholesky, consideremos um caso 3×3 . Para esse cenário, definindo $[L]$ como

$$[L] = \begin{pmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix}, \quad (37)$$

temos que

$$[L][L]^T = \begin{pmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix} \cdot \begin{pmatrix} \ell_{11} & \ell_{21} & \ell_{31} \\ 0 & \ell_{22} & \ell_{32} \\ 0 & 0 & \ell_{33} \end{pmatrix} \quad (38)$$

Realizando a multiplicação matricial, temos:

$$[L][L]^T = \begin{pmatrix} \ell_{11}^2 & \ell_{11}\ell_{21} & \ell_{11}\ell_{31} \\ \ell_{11}\ell_{21} & \ell_{21}^2 + \ell_{22}^2 & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} \\ \ell_{11}\ell_{31} & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} & \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 \end{pmatrix}. \quad (39)$$

É fácil perceber que o produto $[L][L]^T$ nesse caso resulta em uma matriz simétrica, de tal sorte que se $[A] = [A]^T$, podemos então decompor $[A]$ em $[A] = [L][L]^T$. Além disso, para esse cenário 3×3 podemos escrever as seguintes relações entre os termos de $[A]$ e os termos da matriz $[L]$:

$$\begin{aligned} a_{11} &= \ell_{11}^2, & a_{12} &= a_{21} = \ell_{11}\ell_{21}, & a_{13} &= a_{31} = \ell_{11}\ell_{31} \\ a_{22} &= \ell_{21}^2 + \ell_{22}^2, & a_{23} &= a_{32} = \ell_{21}\ell_{31} + \ell_{22}\ell_{32}, & a_{33} &= \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2. \end{aligned} \quad (40)$$

As relações (40) podem ser invertidas, reorganizadas e generalizadas para obtenção das componentes de $[L]$ em sistemas de ordem n em termos das seguintes relações de recorrência:

$$\begin{aligned} \ell_{ki} &= \frac{a_{ki} - \sum_{j=1}^{i-1} \ell_{ij}\ell_{kj}}{\ell_{ii}} \rightarrow \text{para } i = 1, 2, \dots, k-1 \\ \ell_{kk} &= \sqrt{a_{kk} - \sum_{j=1}^{k-1} \ell_{kj}^2} \end{aligned} \quad (41)$$

As relações (41) representam o que conhecemos como decomposição de Cholesky. Após o processo de decomposição é necessário resolver o sistema linear. Isso é feito assumindo que se $[A] = [L][L]^T$, então o sistema linear é dado por $[L][L]^T\{x\} = \{b\}$. Chamando $[L]^T\{x\} = \{y\}$, podemos reescrever esse sistema em termos de duas equações:

$$[L]\{y\} = \{b\} \quad \text{e} \quad [L]^T\{x\} = \{y\}. \quad (42)$$

Como as duas equações em (42) envolvem apenas matrizes triangulares podemos utilizar substituições progressivas e regressivas para calcular $\{y\}$ e $\{x\}$ respectivamente.

O método de Gauss-Seidel

Todos os métodos vistos até aqui são métodos que organizam a solução do sistema sem a necessidade de processos iterativos. Em outras palavras, os métodos que possuem sua origem na ideia da eliminação Gaussiana, não demandam um chute inicial da solução que vai sendo gradativamente refinado por meio de um processo iterativo. Nessa sessão vamos introduzir um esquema iterativo muito utilizado no campo da solução de sistemas lineares, conhecido como método de Gauss-Seidel. Para compreendermos a lógica do método de Gauss-Seidel, começemos com um sistema 3×3 do tipo:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \quad (43)$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2 \quad (44)$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3. \quad (45)$$

Isolando x_1, x_2, x_3 da primeira, segunda e terceira equação que compõe o sistema, temos

$$x_1 = (b_1 - a_{12}x_2 - a_{13}x_3) / a_{11} \quad (46)$$

$$x_2 = (b_2 - a_{21}x_1 - a_{23}x_3) / a_{22} \quad (47)$$

$$x_3 = (b_3 - a_{31}x_1 - a_{32}x_2) / a_{33}. \quad (48)$$

Esse sistema pode ser transformado em um sistema iterativo por meio de duas formas diferentes. A primeira delas é ilustrada a seguir:

$$x_1^{j+1} = (b_1 - a_{12}x_2^j - a_{13}x_3^j) / a_{11} \quad (49)$$

$$x_2^{j+1} = (b_2 - a_{21}x_1^{j+1} - a_{23}x_3^j) / a_{22} \quad (50)$$

$$x_3^{j+1} = (b_3 - a_{31}x_1^{j+1} - a_{32}x_2^{j+1}) / a_{33}, \quad (51)$$

em que $j = 0, \dots, N_{iter}$ sendo N_{iter} o número de iterações. Note que nas equações (49-51) a partir do momento em que obtemos uma melhor aproximação para um dos termos de $\{x\}$ imediatamente inserimos o novo valor na estimativa do próximo termo da solução que buscamos. Essa é a essência do método de Gauss-Seidel. Uma outra opção para a montagem do sistema iterativo é apresentada a seguir:

$$x_1^{j+1} = (b_1 - a_{12}x_2^j - a_{13}x_3^j) / a_{11} \quad (52)$$

$$x_2^{j+1} = (b_2 - a_{21}x_1^j - a_{23}x_3^j) / a_{22} \quad (53)$$

$$x_3^{j+1} = (b_3 - a_{31}x_1^j - a_{32}x_2^j) / a_{33}. \quad (54)$$

As equações (52-54) partem de um conjunto de pontos iniciais e usam esse conjunto para evoluir em bloco a solução para o próximo ponto. Esse esquema é conhecido como iteração de Jacobi e constitui uma variante do método de Gauss-Seidel. Como a proposta do método de Gauss-Seidel já considera o último valor atualizado de uma determinada componente do vetor solução que buscamos na estimativa da próxima, essa é a escolha dominante no campo de esquemas numéricos iterativos.

Como todo método iterativo, o método de Gauss-Seidel só será convergente em certas condições. A condição principal de convergência do método é dada por:

$$|a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}|. \quad (55)$$

A equação acima mostra que Gauss-Seidel é convergente para matrizes com dominância diagonal. Uma forma interessante de controlarmos a convergência do método inclui uma estratégia chamada *relaxação*. Nesse contexto, após o cálculo do valor atual de x_{i+1} pelo método de Gauss-Seidel convencional, aplicamos a seguinte correção

$$x_{i+1} = \lambda x_{i+1} + (1 - \lambda)x_i. \quad (56)$$

Nesse contexto, temos três possibilidades:

- Se $\lambda = 1$, temos o método de Gauss-Seidel sem relaxação original;
- Se $0 < \lambda < 1$, temos o método de Gauss-Seidel com sub-relaxamento. Essa estratégia ajuda alguns sistemas não convergentes a convergirem;
- Se $1 < \lambda < 2$, temos o método de Gauss-Seidel com sobre-relaxamento ou SOR. Essa estratégia acelera a convergência de alguns sistemas de lenta convergência ao dar mais peso para os valores mais recentes de x , assumindo aqui que a solução está caminhando no rumo certo;

Atividade para Casa #3 (APC)

Nesta atividade, o objetivo é desenvolver a compreensão analítica de ideias e conceitos

1. Para a matriz $[A]$ abaixo, calcule a inversa de $[A]$ utilizando a técnica decorrente da decomposição L.U e em seguida verifique de $[A][A]^{-1} = [I]$;

$$[A] = \begin{pmatrix} 3 & -0.1 & -0.2 \\ 0.1 & 7 & -0.3 \\ 0.3 & -0.2 & 10 \end{pmatrix} \quad (57)$$

2. Considere o seguinte sistema tridiagonal:

$$\begin{pmatrix} 2.04 & -1 & 0 & 0 \\ -1 & 2.04 & -1 & 0 \\ 0 & -1 & 2.04 & -1 \\ 0 & 0 & -1 & 2.04 \end{pmatrix} \cdot \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{Bmatrix} = \begin{Bmatrix} 40.8 \\ 0.8 \\ 0.8 \\ 200.8 \end{Bmatrix} \quad (58)$$

Resolva esse sistema na mão, aplicando a técnica de decomposição L.U como foi apresentada. Em seguida, monte uma tabela e execute na mão os laços apresentados no pseudocódigo vinculado ao algoritmo de Thomas. Compare os procedimentos em termos do número de operações de ponto flutuante e do resultado final obtido a partir de cada caminho.

3. Invente um sistema linear se sua escolha no qual a matriz dos coeficientes seja uma matriz quadrada, simétrica, de ordem 3. Você pode escolher os números que quiser para compor a matriz dos coeficientes e o vetor $\{b\}$, desde que a matriz $[A]$ seja simétrica. Em seguida, aplique as relações de recorrência associadas à decomposição de Cholesky e resolva o sistema linear proposto.
4. Use o método de Gauss-Seidel (a) sem relaxamento e (b) com relaxamento (utilizando $\lambda = 1.2$) para resolver o seguinte sistema de equações com um erro relativo de 5%. Se necessário, reorganize as equações para garantir a convergência.

$$\begin{aligned} 2x_1 - 6x_2 - x_3 &= -38 \\ -3x_1 - x_2 + 7x_3 &= -34 \\ -8x_1 + x_2 - 2x_3 &= -20 \end{aligned}$$

1. Mostre que a substituição da solução proposta na equação do movimento leva a um problema algébrico cuja solução depende da resolução de um polinômio característico de ordem $2N$ na variável λ ;

Observação: Esta atividade deve ser desenvolvida de forma analítica (papel e lápis), podendo também ser digitada em $\text{\LaTeX}$ e entregue em formato PDF (preferencialmente), ou escaneada a partir de anotações feitas no papel, desde que o estudante possua uma caligrafia legível e uma organização clara na apresentação das ideias.

Programa para Casa #3 (PPC)

A solução exata do problema de distribuição de temperatura em uma parede plana no contexto 1D, em regime transiente, sem geração interna de energia, a partir de condições de contorno de convecção nas extremidades livres e com simetria adiabática é dada por

$$\theta(x^*, Fo) = \sum_{n=1}^{\infty} \left[\frac{4 \sin(\lambda_n)}{2\lambda_n + \sin(2\lambda_n)} \right] \cos(\lambda_n x^*) e^{-\lambda_n^2 Fo} \quad \text{com} \quad \lambda_n = Bi \cot(\lambda_n), \quad (59)$$

em que θ , x^* e Fo representam respectivamente as versões não-dimensionais de temperatura, posição espacial e instante de tempo e Bi é o número de Biot. Esses termos são dados por

$$\theta = \frac{T - T_{\infty}}{T_i - T_{\infty}}, \quad x^* = \frac{x}{L}, \quad Fo = \frac{\alpha t}{L^2} \quad \text{e} \quad Bi = \frac{hL}{k} \quad (60)$$

Essa solução vem da aplicação da técnica de separação de variáveis aplicada à seguinte EDP

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad (61)$$

em que T representa o campo de temperatura, nesse caso uma função do espaço x e do tempo t e α é a difusividade térmica do material. A demonstração da solução da equação (61) é fornecida nas aulas abaixo, disponibilizadas gratuitamente no canal do YouTube Ciência e brisa:

- Introdução à condução de calor em regime transiente
- Condução transiente - Parte 2

No contexto em que a parede encontra-se sujeita a um processo de geração interna, onde existe uma taxa de geração de energia interna, proveniente de outras fontes, como por exemplo de processos de fissão nuclear, a equação governante do problema possui um termo adicional e é dada por

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} + \frac{\dot{q}}{\rho C_p}, \quad (62)$$

em que $\dot{q}$ representa a taxa de geração interna, ρ é a massa específica do material que constitui a parede e C_p é o calor específico à pressão constante.

Vamos considerar então o problema de modelagem da distribuição de temperatura no espaço e no tempo no interior de um combustível sólido de um reator nuclear, como ilustrado na figura (2). Nesse contexto, h representa o coeficiente de transferência de calor por convecção associado ao escoamento de um fluido externo que com temperatura T_{∞} tenta resfriar o reator. Agora, devido à presença de um termo fonte adicional na equação (62) não é mais possível a obtenção de uma solução analítica por separação de variáveis como a que propusemos para o caso sem geração.

A estratégia de solução numérica do problema passa então pela discretização do domínio de cálculo numa rede nodal de diferenças finitas, ilustrada esquematicamente pela figura (3). Dentro dessa rede, podemos aproximar as derivadas parciais associadas ao tempo e ao espaço pelas expressões abaixo

$$\frac{\partial T}{\partial t} \approx \frac{T_m^{p+1} - T_m^p}{\Delta t} \quad (63)$$

$$\left(\frac{\partial T}{\partial x}\right)_{m+\frac{1}{2}} \approx \frac{T_{m+1} - T_m}{\Delta x} \quad \text{e} \quad \left(\frac{\partial T}{\partial x}\right)_{m-\frac{1}{2}} \approx \frac{T_m - T_{m-1}}{\Delta x} \quad (64)$$

$$\left(\frac{\partial^2 T}{\partial x^2}\right)_m \approx \frac{\partial T / \partial x|_{m+1/2} - \partial T / \partial x|_{m-1/2}}{\Delta x} = \frac{T_{m+1} + T_{m-1} - 2T_m}{\Delta x^2} \quad (65)$$

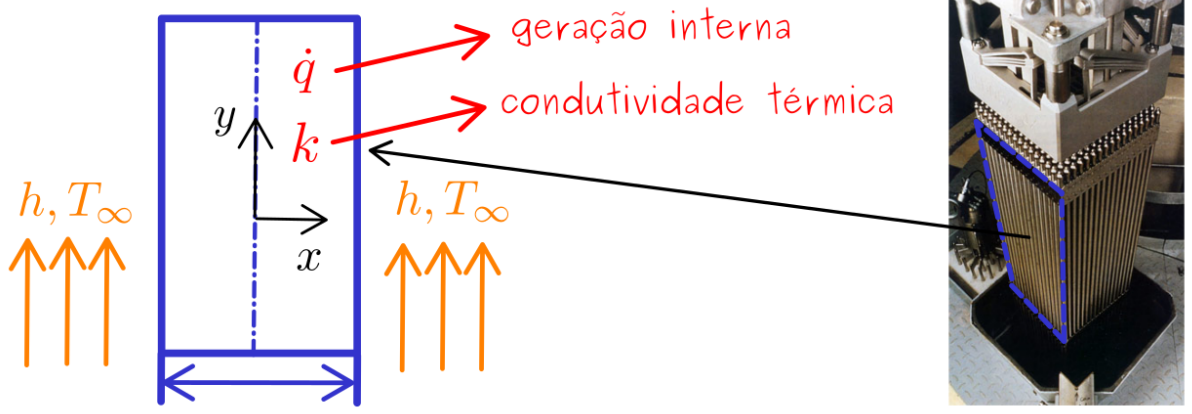


Figura 2: Geometria simplificada e condições de contorno do problema proposto

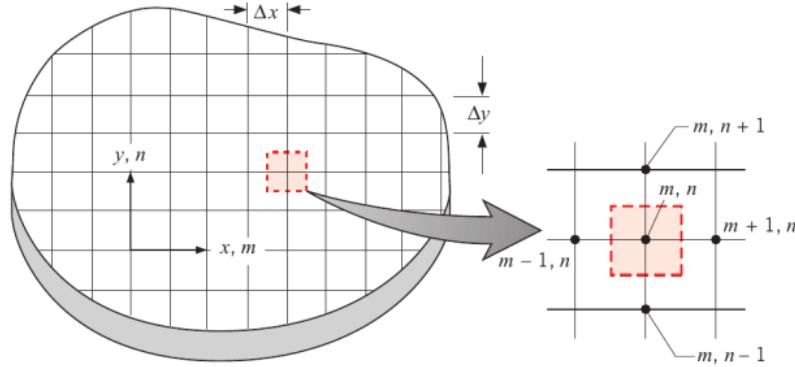


Figura 3: Esquema ilustrativo de uma rede nodal de diferenças finitas

Aplicando as equações (63), (64) e (65) sobre a equação (62) é possível obter a versão discretizada de nossa equação governante, dada por

$$\frac{T_m^{p+1} - T_m^p}{\Delta t} = \alpha \left(\frac{T_{m+1}^Q + T_{m-1}^Q - 2T_m^Q}{\Delta x^2} \right) + \frac{\dot{q}}{\rho C_p}. \quad (66)$$

Temos aqui três opções de escolha para o instante Q no qual iremos avaliar os termos do lado direito da equação (66):

- Opção 1 $\rightarrow Q = p \rightarrow$ esquema explícito;
- Opção 2 $\rightarrow Q = p + 1 \rightarrow$ esquema implícito;
- Opção 3 $\rightarrow p < Q < p + 1 \rightarrow$ esquema semi-implícito;

No esquema explícito a solução parte de um ponto de partida (condição inicial) e evolui por substituição progressiva. Nesse esquema não há necessidade de resolvermos um sistema linear. Essa solução pode no entanto ser instável para maiores passos de tempo. Já o esquema implícito acopla a informação num dado nó com seus vizinhos instantaneamente, o que demanda que o problema seja resolvido como um sistema linear. Esse é um método mais estável numericamente e aceita maiores passos de tempo quando comparado com o esquema explícito. Finalmente, o esquema semi-implícito demanda a solução de um sistema linear, assim como o método implícito, mas permite a inclusão de um parâmetro numérico para controle da convergência do sistema. Para ficar mais claro como trabalhamos o método semi-implícito, vamos reescrever nossa equação discretizada como:

$$T_m^{p+1} - T_m^p = Fo^{\Delta t} (T_{m+1}^Q + T_{m-1}^Q - 2T_m^Q) + A \rightarrow \text{com } Fo^{\Delta t} = \frac{\alpha \Delta t}{\Delta x^2} \text{ e } A = \frac{\dot{q}}{\rho C_p} \quad (67)$$

No esquema semi-implícito, temos

$$T_m^{p+1} - T_m^p = Fo^{\Delta t} [\beta (T_{m+1}^p + T_{m-1}^p - 2T_m^p) + (1 - \beta) (T_{m+1}^{p+1} + T_{m-1}^{p+1} - 2T_m^{p+1})] + A, \quad (68)$$

em que β é um parâmetro numérico de controle, onde

$$\begin{aligned} \beta = 0 &\rightarrow \text{esquema implícito} \\ \beta = 1 &\rightarrow \text{esquema explícito} \\ 0 < \beta < 1 &\rightarrow \text{esquema semi-implícito} \\ \beta = \frac{1}{2} &\rightarrow \text{método de Crank-Nicholson} \end{aligned}$$

Vamos agora ilustrar como podemos usar essas ideias para resolver o problema de distribuição de temperatura no interior do nosso combustível sólido de reator nuclear para uma malha pequena, onde a construção do sistema linear resultante poderá ser vista sem a necessidade ainda de escrevermos um programa de computador para isso. Para isso, vamos considerar uma malha simples de 7 nós, como ilustrado na figura (4).

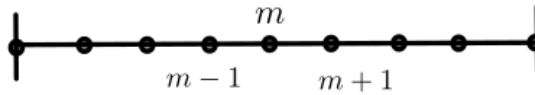


Figura 4: Malha 1D de diferenças finitas

Na nossa malha 1D de diferenças finitas, os nós 1 e 7 são nós de contorno, enquanto os nós 2, 3, 4, 5 e 6 são nós internos. O ponto de partida aqui consiste na organização das equações discretizadas para os nós internos. Vamos para isso considerar a utilização do esquema implícito. Os nós de contorno serão tratados posteriormente, no momento da aplicação das condições de contorno. A aplicação da equação (68) para os nós 2, 3, 4, 5 e 6, considerando o esquema implícito fornece as seguintes equações para cada nó

$$\begin{aligned} 2 &\rightarrow (1 + 2Fo^{\Delta t})T_2^{p+1} - Fo^{\Delta t}(T_1^{p+1} + T_3^{p+1}) = T_2^p + A \\ 3 &\rightarrow (1 + 2Fo^{\Delta t})T_3^{p+1} - Fo^{\Delta t}(T_2^{p+1} + T_4^{p+1}) = T_3^p + A \\ 4 &\rightarrow (1 + 2Fo^{\Delta t})T_4^{p+1} - Fo^{\Delta t}(T_3^{p+1} + T_5^{p+1}) = T_4^p + A \\ 5 &\rightarrow (1 + 2Fo^{\Delta t})T_5^{p+1} - Fo^{\Delta t}(T_4^{p+1} + T_6^{p+1}) = T_5^p + A \\ 6 &\rightarrow (1 + 2Fo^{\Delta t})T_6^{p+1} - Fo^{\Delta t}(T_5^{p+1} + T_7^{p+1}) = T_6^p + A. \end{aligned} \quad (69)$$

Para os nós de contorno, podemos aplicar um balanço de energia para cada um deles, começando pelo nó 7. Esse balanço é explicitado na figura (5)

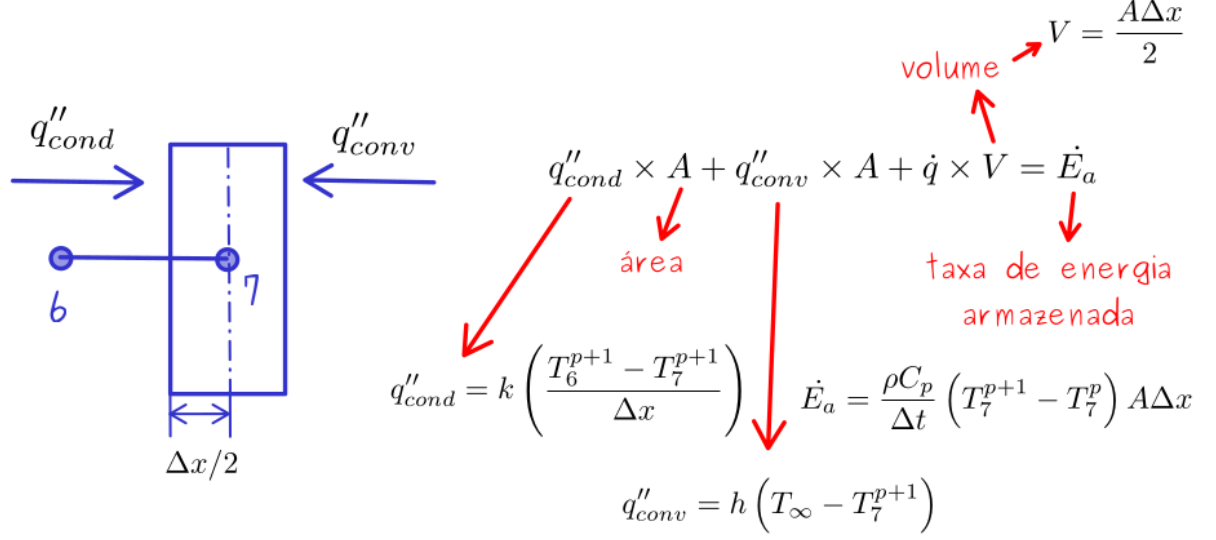


Figura 5: Balanço de energia para o nó 7 do nosso problema

Equacionando todos os termos do balanço de energia para o nós 7 e reorganizando os termos, obtemos

$$(1 + 2Fo^{\Delta t} + 2Fo^{\Delta t}Bi)T_7^{p+1} - 2Fo^{\Delta t}T_6^{p+1} = 2Fo^{\Delta t}BiT_\infty + \frac{Fo^{\Delta t}\dot{q}\Delta x^2}{k} + T_7^p. \quad (70)$$

Para o nó 1, a análise é mais simples, pois assumimos simetria adiabática, o que equivale à pensar nesse nó como se fosse um nó interno que possui o mesmo vizinho pela esquerda e pela direita. Aplicando esse raciocínio para a equação geral de um nó interno qualquer, temos

$$(1 + 2Fo^{\Delta t})T_1^{p+1} - 2Fo^{\Delta t}T_2^{p+1} = T_1^p + A. \quad (71)$$

Finalmente, podemos escrever todas essas equações na forma de um sistema linear $[A]\{T\} = \{b\}$ com

$$[A] = \begin{pmatrix} 1 + 2Fo & -2Fo & 0 & 0 & 0 & 0 & 0 \\ -Fo & 1 + 2Fo & -Fo & 0 & 0 & 0 & 0 \\ 0 & -Fo & 1 + 2Fo & -Fo & 0 & 0 & 0 \\ 0 & 0 & -Fo & 1 + 2Fo & -Fo & 0 & 0 \\ 0 & 0 & 0 & -Fo & 1 + 2Fo & -Fo & 0 \\ 0 & 0 & 0 & 0 & -Fo & 1 + 2Fo & -Fo \\ 0 & 0 & 0 & 0 & 0 & -2Fo & 1 + 2Fo + 2FoBi \end{pmatrix}$$

e

$$\{b\} = \begin{pmatrix} T_1^p + A \\ T_2^p + A \\ T_3^p + A \\ T_4^p + A \\ T_5^p + A \\ T_6^p + A \\ T_7^p + Fo \left(2BiT_\infty + \frac{\dot{q}\Delta x^2}{k} \right) \end{pmatrix}$$

Na composição da matriz $[A]$ explicitada acima, o símbolo Fo equivale ao que definimos como $Fo^{\Delta t}$. A partir dessa contextualização, você deverá desenvolver as seguintes atividades de programação.

1. Escreva primeiramente um programa que resolva numericamente o problema, usando o método das diferenças finitas para o caso sem geração interna e valide sua solução numérica com base em uma comparação com a solução exata. Utilize o algoritmo de Thomas para sistemas tridiagonais na hora de resolver o seu problema e um esquema implícito de evolução no tempo;
2. Em seguida, acrescente o termo de geração interna à solução solução numérica, usando como base a malha e passo de tempo ajustados no passo anterior para validação da solução no limite assintótico em que $\dot{q} \rightarrow 0$;
3. Teste diferentes formas de representação visual da sua solução (perfis, campos de cores, animações, etc.);
4. Pesquise valores realistas para o problema, tendo em vista o contexto de aplicação aqui desenvolvido;

Requisitos de implementação:

- O código deve ser implementado a partir dos conceitos desenvolvidos pelo aluno, não sendo permitido o uso de bibliotecas que realizem diretamente a integração numérica;
- É permitido o uso de estruturas básicas da linguagem, como vetores, matrizes, funções, laços e condicionais;
- O código deve estar devidamente comentado e organizado;
- Os resultados devem ser apresentados de forma clara, incluindo gráficos e análises.

Entrega:

- O código deverá ser enviado pelo ambiente Aprender3;
- O programa deverá ser disponibilizado em repositório próprio no GitHub, acompanhado de documentação adequada (README.md);
- A documentação deverá permitir que terceiros compreendam, executem e reproduzam os resultados obtidos.

Referências bibliográficas

1. S. C. Chapra, R. P. Canale. “Métodos Numéricos para Engenharia.”, McGrawHill, 5a edição (2008): 1-825.
2. R. G. Gontijo, “Notas de aula do curso de Cálculo Numérico Aplicado”, (2026).